



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

580506004-8

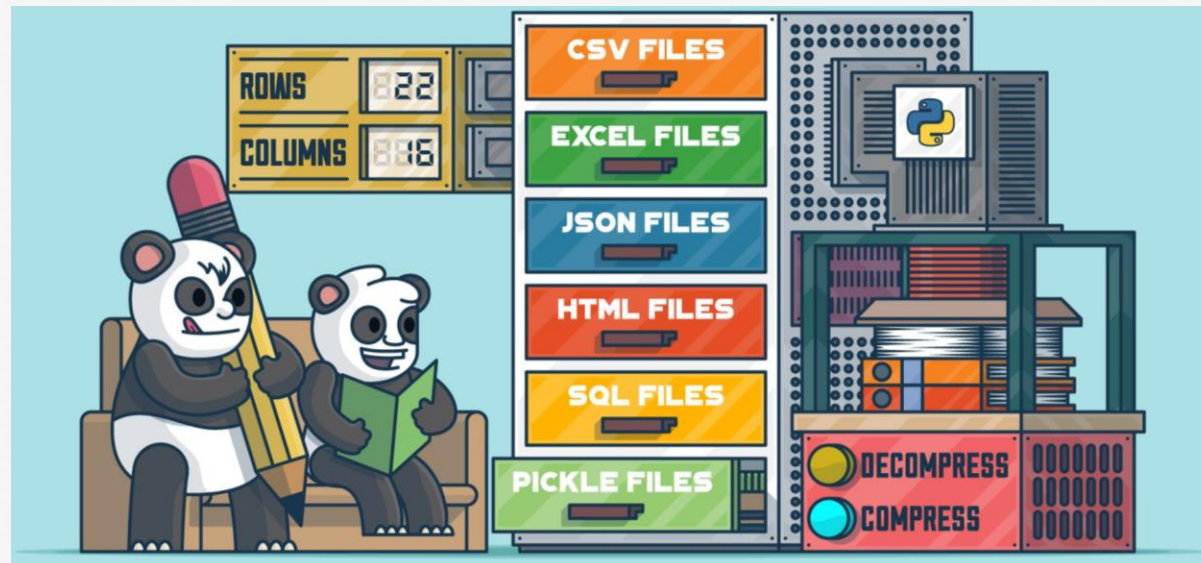
William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



DATAFRAMES

Los *DataFrames* son una estructura de datos *bidimensional* muy utilizada en ciencia de datos .

La biblioteca más comúnmente utilizada para trabajar con DataFrames en Python es *pandas*.

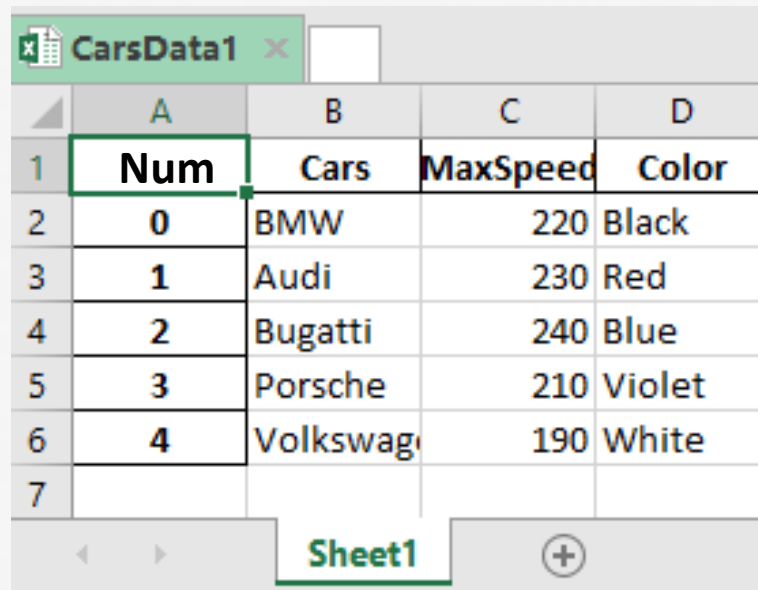


¿Qué son los DataFrames?

Un DataFrame es esencialmente una *tabla de datos*, similar a una hoja de cálculo en Excel.

Está diseñado para almacenar datos en forma de *filas* y *columnas*.

Los DataFrames proporcionan una forma estructurada de organizar y analizar datos tabulares.



	A	B	C	D
1	Num	Cars	MaxSpeed	Color
2	0	BMW	220	Black
3	1	Audi	230	Red
4	2	Bugatti	240	Blue
5	3	Porsche	210	Violet
6	4	Volkswagen	190	White
7				

DATAFRAMES

- Cada columna representa una *variable* o una *característica*
- Cada fila representa una *observación* o un *dato*
- Tanto las filas como columnas tienen una *etiqueta*

Numero paciente	Nombre	Edad	Ocupación
1	Carlos	34	Médico
2	Marta	30	Secretaria
3	Andrea	15	Estudiante
4	Jorge	27	Policía



Institución
Universitaria
Reacreditada en Alta Calidad

DATAFRAMES

En Python, para definir un dataframe, en primer lugar necesitamos importar el módulo pandas.

```
import pandas as pd
```



Institución
Universitaria
Reacreditada en Alta Calidad

DATAFRAMES – CÓMO CREARLOS

Podemos crear un dataframe a partir de: un diccionario, una lista de listas, y una lista de diccionarios.

DATAFRAMES – CÓMO CREARLOS

Usando Diccionarios : primero creamos un diccionario donde las **claves** serán los nombres de las **columnas** y los **valores** serán listas, con tantos elementos como número de **filas** queramos.

Ejemplo: Dataframe con 5 filas y 2 columnas.

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10

→ **Claves**

Listas

DATAFRAMES – CÓMO CREARLOS

```
data = {"x": [1, 2, 3, 4, 5], "y": [2, 4, 6, 8, 10]}
```

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10

→ **Claves**

Listas

```
df1 = pd.DataFrame(data = data)  
print(df1)
```

DATAFRAMES – CÓMO CREARLOS

NOTA: hay una columna adicional de 5 números ordenados verticalmente del 0 al 4.

Se trata simplemente del nombre de cada fila, que por defecto es el índice de cada fila. El 0 indica la primera fila; el 1, la segunda; y así, sucesivamente.

DATAFRAMES – CÓMO CREAMOS

Usando Listas : En este caso, podemos hacerlo directamente con la función DataFrame() de pandas, usando los parámetros data y columns.

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10

→ **Lista
columns**

**Lista de
listas data**

```
df2 = pd.DataFrame(data = [[1, 2], [2, 4], [3, 6], [4, 8], [5, 10]], columns = ["x", "y"])  
print(df2)
```

DATAFRAMES – CAMBIAR EL NOMBRE DE LAS FILAS

```
data = {"x": [1, 2, 3, 4, 5], "y": [2, 4, 6, 8, 10]}
```

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10



	x	y
obs1	1	2
obs2	2	4
obs3	3	6
obs4	4	8
obs5	5	10

```
df3 = pd.DataFrame(data = data, index = ["obs1", "obs2", "obs3", "obs4", "obs5"])  
print(df3)
```

DATAFRAMES – CREAR UN DATAFRAME CON MENOS COLUMNAS

```
d = {"a": [1, 2, 3], "b": [4, 5, 6], "b1": [7, 8, 9]}
```

los nombres de las columnas indicados deben ser iguales a las claves del diccionario

	a	b
0	1	4
1	2	5
2	3	6

```
df = pd.DataFrame(d, columns = ["a", "b"])  
print(df)
```

DATAFRAMES – CÓMO CREARLOS

Usando una lista de diccionarios : En este caso se debe tener cuidado con las claves a usar. Se recomienda que sean todas iguales

```
data = [{"x": 1, "y": 2},  
        {"x": 2, "y": 4},  
        {"x": 3, "y": 6},  
        {"x": 4, "y": 8},  
        {"x": 5, "y": 10}]
```

```
df4 = pd.DataFrame(data = data)  
print(df4)
```

DATAFRAMES – CÓMO CREARLOS

Usando una lista de tuplas o la función zip : Para ello, a partir de dos listas, creamos una lista de tuplas, que es la que proporcionamos al parámetro data de la función DataFrame() para construir el dataframe.

```
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

data = list(zip(x, y))
print(data)
```

```
df5 = pd.DataFrame(data, columns = ["x", "y"])
print(df5)
```

DATAFRAMES – CÓMO CREARLOS

Usando el método .from_dict y diccionarios

```
d = {"a": [1, 2, 3],  
     "b": [4, 5, 6],  
     "b1": [7, 8, 9]}
```

```
df = pd.DataFrame.from_dict(d)  
print(df)
```

DATAFRAMES – CÓMO CREAMOS

El método `.from_dict` nos permite utilizar las claves como nombres de las filas, en este caso debemos darle un nombre a cada columna

```
d = {"fila1": [1, 4, 7],  
     "fila2": [2, 5, 8],  
     "fila3": [3, 6, 9]}
```

```
df = pd.DataFrame.from_dict(d, orient = "index", columns = ["A", "B", "C"])  
print(df)
```

DATAFRAMES – TAMAÑO

Con el método `.shape` podemos calcular las dimensiones (número de filas y columnas) del dataframe

`df.shape`

DATAFRAMES – TAMAÑO

Como resultado obtenemos una tupla donde el primer elemento es el número de filas, mientras que el segundo elemento es el número de columnas, podemos acceder a sus elementos de la siguiente manera:

```
num_filas = df.shape[0]  
num_columnas = df.shape[1]  
print("El número de filas de df es", num_filas)  
print("El número de columnas de df es", num_columnas)
```

DATAFRAMES – CANTIDAD DE ELEMENTOS

Con el método `.size` calculamos el número total de valores que tiene el dataframe (número de filas por número de columnas)

```
df.size
```

ACTIVIDAD

Cree el siguiente dataframe usando tres métodos diferentes:

- A partir de listas
- A partir de un diccionario
- A partir de una lista de tuplas (función zip)

	Nombre	Edad	Calificaciones
0	Juan	22	4.2
1	María	24	4.5
2	Pedro	21	3.9
3	Ana	23	4.6
4	Luis	25	4.4

SUBDATAFRAMES

Dado un dataframe, un subdataframe no es más que la selección de unas filas y columnas en particular.

Dataframe

Número	Nombre	Edad	Profesión
1	Antonio	43	Cantante
2	Jorge	52	Abogado
3	Alberto	35	Médico
4	Jaime	55	Ingeniero

Subdataframe

Número	Nombre	Profesión
1	Antonio	Cantante
3	Alberto	Médico

SUBDATAFRAMES – Selección a partir de columnas

Creemos primero el siguiente dataframe:

```
fdata = {"Name": ["Alicia", "Bill", "Carlos", "Diana"],  
        "Age": [22, 28, 19, 34],  
        "Pet": [True, False, False, True],  
        "Height": [157, 190, 175, 164],  
        "Birthday": ["Mayo", "Junio", "Agosto", "Diciembre"]}  
df = pd.DataFrame(data = fdata, index = ["obs1", "obs2", "obs3", "obs4"])
```

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna por su nombre

```
sub_dataframe = df["Birthday"]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna por su nombre

```
sub_dataframe = df[["Name", "Age"]]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna con el método `.columns[ ]`

```
sub_dataframe=df[df.columns[4]]  
  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna con el método `.columns[ ]`

```
sub_dataframe=df[df.columns[[0, 1]]]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna con el método `.columns[ ]`

```
sub_dataframe=df[df.columns[2:4]]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna con el método `.loc[]` , este nos permite hacer un rebanado

```
sub_dataframe = df.loc[:, "Birthday"]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna con el método `.loc[]` , este nos permite hacer un rebanado

```
sub_dataframe = df.loc[:, ["Age", "Pet"]]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna con el método `.loc[]` , este nos permite hacer un rebanado

```
sub_dataframe = df.loc[:, "Pet":"Birthday"]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna con el método `.iloc[]` ,
funciona a partir de un índice

```
sub_dataframe = df.iloc[ : , 4 ]  
  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna con el método `.iloc[]` ,
funciona a partir de un índice

```
sub_dataframe = df.iloc[:, [0, 2, 4]]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de columnas

Seleccionamos una columna con el método `.iloc[]` ,
funciona a partir de un índice

```
sub_dataframe = df.iloc[:, 0:2]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de filas

Retomemos el dataframe previamente creado:

```
fdata = {"Name": ["Alicia", "Bill", "Carlos", "Diana"],  
        "Age": [22, 28, 19, 34],  
        "Pet": [True, False, False, True],  
        "Height": [157, 190, 175, 164],  
        "Birthday": ["Mayo", "Junio", "Agosto", "Diciembre"]}  
df = pd.DataFrame(data = fdata, index = ["obs1", "obs2", "obs3", "obs4"])
```

SUBDATAFRAMES – Selección a partir de filas

Seleccionamos una fila por su nombre

```
sub_dataframe = df.loc["obs1"]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de filas

Seleccionamos una fila por un índice

```
sub_dataframe = df.iloc[-1]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de filas

Seleccionamos una fila por su nombre

```
sub_dataframe = df.loc[["obs2","obs3"]]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de filas

Seleccionamos una fila por su nombre

```
sub_dataframe = df.loc["obs2":"obs4"]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de filas

Seleccionamos una fila por su índice

```
sub_dataframe = df.iloc[[1, 3]]  
  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de filas

Seleccionamos una fila por su índice

```
sub_dataframe = df.iloc[0:3]  
print(sub_dataframe)
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de filas y columnas a la vez

hay que indicar la fila y la columna

```
print(df.loc["obs2", "Age"])  
print(df.iloc[1, 1])
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de filas y columnas a la vez

```
print(df.loc["obs2":"obs3", ["Name", "Birthday"]])
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Selección a partir de filas y columnas a la vez

```
print(df.iloc[1:3, [0, 4]])
```

	Name	Age	Pet	Height	Birthday
obs1	Alicia	22	True	157	Mayo
obs2	Bill	28	False	190	Junio
obs3	Carlos	19	False	175	Agosto
obs4	Diana	34	True	164	Diciembre

SUBDATAFRAMES – Actividad

Generar el siguiente dataframe y sus subdataframes correspondientes

	Nombre	Edad	Ciudad	País
0	Juan	30	Bogotá	Colombia
1	María	25	Buenos Aires	Argentina
2	Luis	28	Santiago	Chile
3	Ana	22	Ciudad de México	México
4	Pedro	35	Lima	Perú

	Nombre	País
0	Juan	Colombia
1	María	Argentina
2	Luis	Chile
3	Ana	México
4	Pedro	Perú

	Nombre	Edad	Ciudad	País
0	Juan	30	Bogotá	Colombia
1	Luis	28	Santiago	Chile
2	Pedro	35	Lima	Perú

	Nombre	Edad	Ciudad
1	María	25	Buenos Aires
2	Luis	28	Santiago
3	Ana	22	Ciudad de México
4	Pedro	35	Lima

DATAFRAMES – MÉTODOS ÚTILES

El método `.head()` sirve para visualizar las primeras filas del dataframe. Por defecto, se nos mostrarán las 5 primeras.

```
d = {"fruit": ["sandía", "melón", "manzana", "cerezas", "plátano", "pera", "melocotón", "fresas"],  
     "count": [1, 1, 6, 10, 3, 6, 4, 10]}
```

```
df = pd.DataFrame(d)
```

```
df.head()
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.tail()` sirve para visualizar las últimas filas del dataframe. Por defecto, se nos mostrarán las 5 últimas

```
d = {"fruit": ["sandía", "melón", "manzana", "cerezas", "plátano", "pera", "melocotón", "fresas"],  
     "count": [1, 1, 6, 10, 3, 6, 4, 10]}
```

```
df = pd.DataFrame(d)
```

```
df.tail()
```

DATAFRAMES – MÉTODOS ÚTILES

Si no creamos una copia utilizando el método `.copy()` , cualquier modificación en la copia se verá reflejado en el dataframe original

```
fruits = df.copy()
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.rename()` se puede utilizar tanto para cambiar las etiquetas de las filas como los nombres de las columnas. El parámetro `inplace` se usa para asegurar que el cambio se haga.

```
df.rename(columns = {"fruit": "fruta", "count": "cantidad"},inplace = True)  
df
```

DATAFRAMES – MÉTODOS ÚTILES

Con el método `.columns` también podemos cambiar el nombre de las columnas

```
df.columns = ["FRUTA", "CANTIDAD"]  
df
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.rename()` se puede utilizar tanto para cambiar las etiquetas de las filas como los nombres de las columnas. El parámetro `inplace` se usa para asegurar que el cambio se haga.

```
df.rename(index = {0: "obs1", 1: "obs2", 7: "obs8"}, inplace = True)  
df
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.insert()` inserta una nueva columna a un dataframe existente

```
df.insert(loc = 1, column = "PRECIO", value = [2.50, 2.00, 0.35, 0.10, 0.35, 0.20, 0.15, 0.05])  
df
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.insert()` inserta una nueva columna a un dataframe existente

```
df.insert(loc = 1, column = "COLOR", value = "rojo")  
df
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.drop()` nos permite borrar las filas o columnas que indiquemos

```
df_dropped = df.drop(labels = ["obs1", 4], axis = 0 , inplace = True)  
df_dropped
```

	FRUTA	COLOR	PRECIO	CANTIDAD
obs1	sandía	rojo	2.50	1
obs2	melón	rojo	2.00	1
2	manzana	rojo	0.35	6
3	cerezas	rojo	0.10	10
4	plátano	rojo	0.35	3
5	pera	rojo	0.20	6
6	melocotón	rojo	0.15	4
obs8	fresas	rojo	0.05	10

DATAFRAMES – MÉTODOS ÚTILES

El método `.drop()` nos permite borrar las filas o columnas que indiquemos

```
df_dropped = df.drop(labels = ["COLOR", "PRECIO"], axis = 1)  
df_dropped
```

	FRUTA	COLOR	PRECIO	CANTIDAD
obs1	sandía	rojo	2.50	1
obs2	melón	rojo	2.00	1
2	manzana	rojo	0.35	6
3	cerezas	rojo	0.10	10
4	plátano	rojo	0.35	3
5	pera	rojo	0.20	6
6	melocotón	rojo	0.15	4
obs8	fresas	rojo	0.05	10

DATAFRAMES – MÉTODOS ÚTILES

El método `.drop()` nos permite borrar las filas o columnas que indiquemos

```
column_popped = df.pop("COLOR")  
df
```

	FRUTA	COLOR	PRECIO	CANTIDAD
obs1	sandía	rojo	2.50	1
obs2	melón	rojo	2.00	1
2	manzana	rojo	0.35	6
3	cerezas	rojo	0.10	10
4	plátano	rojo	0.35	3
5	pera	rojo	0.20	6
6	melocotón	rojo	0.15	4
obs8	fresas	rojo	0.05	10

DATAFRAMES – MÉTODOS ÚTILES

El método `.rank()` devuelve un ranking.

Si por ejemplo queremos un ranking por fruta, el método `.rank()` nos devolverá una columna de posiciones correspondientes a la posición que ocupa cada fruta si estas son ordenadas alfabéticamente

```
listado_frutas = df["FRUTA"].rank()  
listado_frutas
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.nunique()` devuelve el conteo de cuántos valores únicos hay en cada columna

```
df.nunique()
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.nsmallest()` nos devuelve las `n` filas con menor valor de la columna que indiquemos por parámetro

```
df.nsmallest(3, "PRECIO")
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.largest()` nos devuelve las `n` filas con mayor valor de la columna que indiquemos por parámetro

```
df.nlargest(5, "CANTIDAD")
```

DATAFRAMES – MÉTODOS ÚTILES

El método `.dtypes` nos indica de qué tipo es cada columna del dataframe

```
df.dtypes
```

DATAFRAMES – USO DE CICLOS

Primero creamos el siguiente dataframe

```
d = {"name": ["Juan Gabriel", "María", "Ricardo"],  
     "surname": ["Gomila", "Santos", "Alberich"],  
     "gender": ["m", "f", "m"]}  
  
df = pd.DataFrame(d)  
df
```

DATAFRAMES – USO DE CICLOS

Usamos `.iterrows()` para obtener el índice de cada fila junto al contenido de cada una

```
for i, j in df.iterrows():  
  
    print("Índice de la fila: {},\n\n Contenido de la fila:\n{}".format(i, j), end = "\n\n\n")
```

DATAFRAMES – USO DE CICLOS

Usamos `.itertuples()` para obtener una tupla con toda la información de cada fila

```
for i in df.itertuples():  
    print("Contenido de la fila:\n{}".format(i), end = "\n\n")
```

DATAFRAMES – USO DE CICLOS

Para iterar sobre las columnas de un dataframe, creamos una lista de las columnas del dataframe y luego iteramos sobre esa lista para obtener la información de esas columnas

```
columns = list(df)
print(columns)
```

```
for c in columns:
    print("Columna {}: \n{}".format(c, df[c]), end = "\n\n")
```

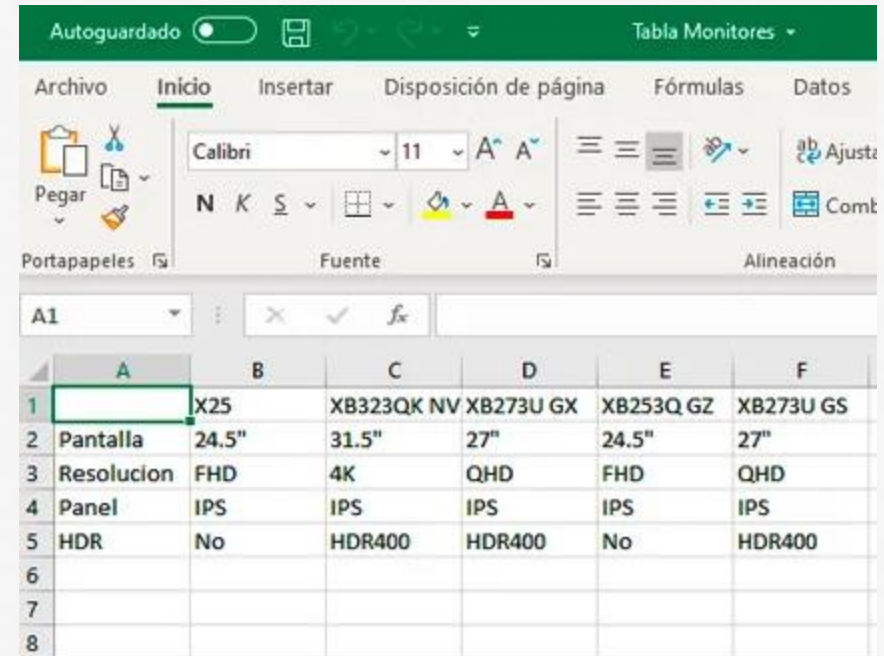
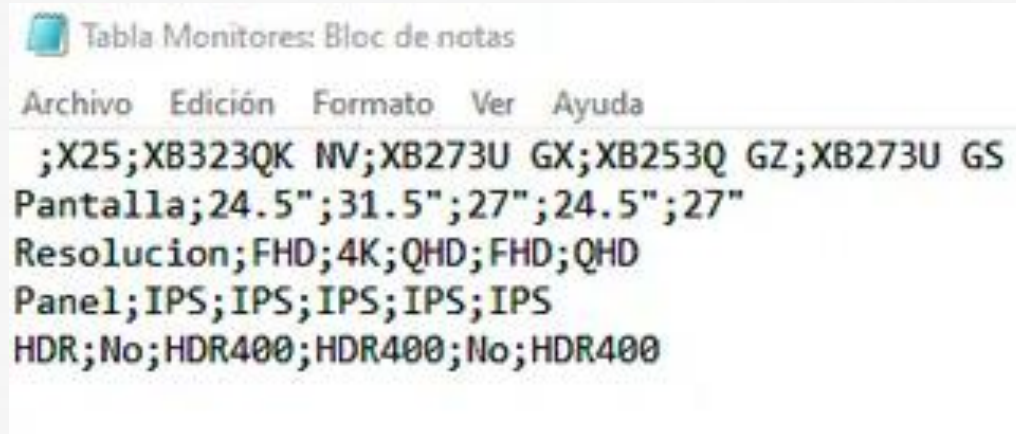
DATAFRAMES – USO DE CICLOS

Usamos `.iteritems()` para obtener el nombre de cada columna junto al contenido de cada una

```
for i, j in df.iteritems():  
  
    print("Nombre de la columna: {},\n\nContenido de la columna:\n{}".format(i, j), end = "\n\n\n")
```

ARCHIVOS CSV

Un archivo CSV (Comma-Separated Values) es un formato de archivo utilizado para almacenar y transmitir datos tabulares de manera simple y legible por máquinas.



Autoguardado    Tabla Monitores ▾

Archivo Inicio Insertar Disposición de página Fórmulas Datos

Calibri 11 A⁺ A⁻            

ARCHIVOS CSV

- Los archivos CSV están compuestos principalmente por *líneas de texto*.
- Cada *línea* representa una *fila* de datos.
- Los valores dentro de cada fila se *separan* por un carácter delimitador, que suele ser una coma (,), aunque también se pueden utilizar otros delimitadores, como punto y coma (;) o tabulaciones.
- La primera fila suele utilizarse para los *encabezados*, donde se describen las columnas y se especifica qué tipo de datos contienen las columnas.



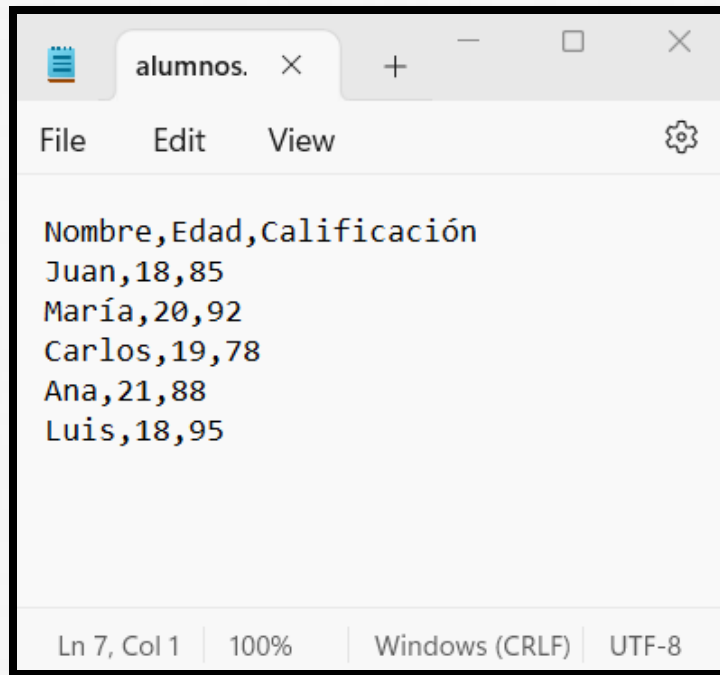
Institución
Universitaria
Reacreditada en Alta Calidad

ARCHIVOS CSV - USOS

- Almacenamiento de datos en hojas de cálculo
- Intercambio de datos entre diferentes aplicaciones y sistemas
- Bases de datos
- Análisis de datos

ARCHIVOS CSV - Ejemplo

- Almacenamiento de datos en hojas de cálculo



alumnos. x + - □ x

File Edit View ⚙

Nombre,Edad,Calificación
Juan,18,85
María,20,92
Carlos,19,78
Ana,21,88
Luis,18,95

Ln 7, Col 1 | 100% | Windows (CRLF) | UTF-8

	A	B	C
1	Nombre ▼	Edad ▼	Calificación ▼
2	Juan	18	85
3	María	20	92
4	Carlos	19	78
5	Ana	21	88
6	Luis	18	95

DE DATAFRAME A ARCHIVO CSV

Podemos convertir un dataframe de Python a un archivo csv,
Inicialmente necesitamos importar las librerías de pandas y
files de Google colab

```
import pandas as pd  
from google.colab import files
```

DE DATAFRAME A ARCHIVO CSV

```
data = {'Nombre': ['Juan', 'María', 'Carlos'],  
        'Edad': [25, 30, 35]}
```

```
df = pd.DataFrame(data)
```

```
nombre_archivo = 'datos.csv'  
df.to_csv(nombre_archivo, index=False)
```

```
files.download(nombre_archivo)
```

DE ARCHIVO CSV A DATAFRAME

Podemos guardar la información de un archivo csv en un dataframe usando la función `read_csv()`.

El archivo puede

- Estar guardado en nuestro directorio de trabajo (Google drive)
- Proceder de una url



DE ARCHIVO CSV A DATAFRAME

Desde el Google Drive

Vamos a cargar el archivo llamado characters-simpsons.csv que tenemos guardado en una carpeta de Google Drive.

Se recomienda crear una llamada 'datasets'

DE ARCHIVO CSV A DATAFRAME

Desde el Google Drive

Para ello, lo primero que tenemos que hacer es conectar este notebook de Colab con Google Drive.

```
from google.colab import drive  
drive.mount('/content/drive')
```

DE ARCHIVO CSV A DATAFRAME

Desde el Google Drive

A continuación, navegamos por nuestro Google Drive hasta encontrar el archivo csv en cuestión. De sus opciones, seleccionamos copiar ruta. Esa ruta será la que pasemos por parámetro a la función `read_csv()`

```
simpsons_df = pd.read_csv(ruta)  
simpsons_df.shape
```



Institución
Universitaria
Reacreditada en Alta Calidad

DATAFRAMES

Tratamiento de datos faltantes

Los datos faltantes se dan cuando no hay información para uno o más elementos.

Éste es un problema muy común en la vida real.

En la librería pandas, los identificamos con valores NA (Not Available) o NaN (Not a Number).

DATAFRAMES

.isnull()

El método `.isnull()` nos devuelve un `True` en donde hay elementos nulos : *`simpsons_df.isnull()`*

	id	name	normalized_name	gender
0	7	Children	children	NaN
1	12	Mechanical Santa	mechanical santa	NaN
2	13	Tattoo Man	tattoo man	NaN
3	16	DOCTOR ZITSOFSKY	doctor zitsofsky	NaN
4	20	Students	students	NaN

	id	name	normalized_name	gender
0	False	False	False	True
1	False	False	False	True
2	False	False	False	True
3	False	False	False	True
4	False	False	False	True

DATAFRAMES

.notnull()

Nos devuelve False allí donde hay un dato faltante:

simpsons_df.notnull()

	id	name	normalized_name	gender
0	7	Children	children	NaN
1	12	Mechanical Santa	mechanical santa	NaN
2	13	Tattoo Man	tattoo man	NaN
3	16	DOCTOR ZITSOFSKY	doctor zitsofsky	NaN
4	20	Students	students	NaN

	id	name	normalized_name	gender
0	True	True	True	False
1	True	True	True	False
2	True	True	True	False
3	True	True	True	False
4	True	True	True	False

DATAFRAMES – Datos faltantes

Pero ... qué hacemos con los elementos NA o NaN ?

Existen muchas técnicas para tratar con valores faltantes: se sustituyen por la media, por la mediana, se elimina la observación, se interpolan...

DATAFRAMES – Datos faltantes

Sustituir valores

El método `.fillna()` sustituye los valores faltantes por el valor que indiquemos por parámetro

```
import numpy as np
```

```
data = {"Primer lanzamiento": [100, 86, np.nan, 75, 97],  
        "Segundo lanzamiento": [80, np.nan, 63, 81, 88],  
        "Tercer lanzamiento": [93, 89, 92, 97, np.nan] }  
  
points_df = pd.DataFrame(data, index = ["Jugador 1", "Jugador 2", "Jugador 3",  
    "Jugador 4", "Jugador 5"])  
  
points_df  
  
points_df.fillna(0)
```

DATAFRAMES – Datos faltantes

Reemplazar valores

Para sustituir valores faltantes con el método `.replace()` lo hacemos del siguiente modo: primero pasamos por parámetro el valor que queremos sustituir y luego, el valor por el cual queremos sustituirlo.

```
points_df = pd.DataFrame(data, index = ["Jugador 1", "Jugador 2",  
"Jugador 3", "Jugador 4", "Jugador 5"])  
  
points_df.replace(np.nan, 0)
```

DATAFRAMES – Datos faltantes

Interpolar valores

Si usamos el método `.interpolate()`, sustituiremos los valores NaN por valores interpolados. Este método consta de muchos parámetros para elegir el método (que por defecto es lineal) por el cual llevar a cabo la interpolación.

```
points_df = pd.DataFrame(data, index = ["Jugador 1", "Jugador 2",  
"Jugador 3", "Jugador 4", "Jugador 5"])  
  
points_df.interpolate()
```

DATAFRAMES – Datos faltantes

Eliminar filas con valores faltantes

El método `.dropna()` elimina las filas que contienen valores faltantes.

```
points_df = pd.DataFrame(data, index = ["Jugador 1", "Jugador 2",  
"Jugador 3", "Jugador 4", "Jugador 5"])  
points_df.dropna()
```

DATAFRAMES – Filtrar por datos

Antes de comenzar vamos a cargar el archivo que contiene la frecuencia de aparición de las letras del alfabeto inglés. Este archivo se llama letter_frequency.csv

```
letters_freq_df =  
pd.read_csv("https://people.sc.fsu.edu/~jburkardt/data/csv/letter_frequency.csv")  
letters_freq_df.columns = ["Letra", "Frecuencia", "Porcentaje"]  
letters_freq_df
```

DATAFRAMES – Filtrar por datos

Dado un dataframe, podemos filtrar sus filas comprobando cuáles satisfacen una condición

Ejemplo: Mostramos las observaciones con porcentaje mayor a 5

```
letters_freq_df[letters_freq_df["Porcentaje"] > 5]
```

DATAFRAMES – Filtrar por datos

Ejemplo: Mostramos las observaciones con frecuencia mayor o igual a la de la letra S

```
freq_S = letters_freq_df.loc[18, "Frecuencia"]  
letters_freq_df[letters_freq_df["Frecuencia"] >= freq_S]
```

DATAFRAMES – Filtrar por datos

El método `.query()` nos puede ser útil para este cometido, pero funciona únicamente cuando los valores de la columna no contienen espacios en blanco

```
letters_freq_df.query('Porcentaje > 5')
```

```
letters_freq_df.query("Porcentaje > 5 and Porcentaje <= 8")
```

DATAFRAMES – Promedio de datos

El método .mean() nos permite calcular la media de los datos de una columna

```
letters_freq_df["Porcentaje"].mean()
```

ACTIVIDAD

Descargue el dataset **Young People Survey** de **Kaggle**

Link: <https://www.kaggle.com/datasets/miroslavsabo/young-people-survey/>

- Cree un nuevo notebook de Google Colab
- Suba el dataset
- Imprima el tamaño del dataset
- Imprima el número de datos
- Cree un subdataset_1 que contenga únicamente 5 géneros de música
- Cree un subdataset_2 que contenga la edad, el peso, la altura y el género
- Cree un subdataset_3 que contenga la mitad de los jóvenes encuestados
- Filtre el dataset con los jóvenes que más le tienen miedo a la oscuridad (le dieron una puntuación de 5)
- Filtre el dataset con los jóvenes que tienen más de 18 años



Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a stylized white 'C' shape on the right side.